

How to Earn Full Collaboration Credit (GitHub Desktop Workflow)

This course uses GitHub the way professional software teams do.

Your collaboration grade is based on **observable GitHub activity**, not self-reports.

If you follow the steps below, you will earn **full collaboration points**.

What is a Pull Request (PR)?

A **pull request (PR)** is a way to propose changes from your branch to the main branch of the repository. It allows your partner (or instructor) to review your work before it is merged. PRs are a core part of collaborative software development and are used to: - Share your changes with your team - Request review and feedback - Keep the main branch stable and organized

All substantial work should be merged into `main` via PRs, and **reviews must be done using PRs**.

1. Individual Contribution (40 pts)

Goal: Show that *you* wrote code.

Do this using GitHub Desktop

- Open GitHub Desktop and **clone the course repository** to your computer.
- Create a new branch via the **Branch → New Branch** menu.
- Make **multiple meaningful commits** locally by editing files and clicking **Commit to <branch>**.
- Push your branch to GitHub using the **Push origin** button.
- Open a **pull request** on GitHub via the **Current Branch → Create Pull Request** button.

Avoid

- One giant commit at the end
 - Letting your partner commit `your work`
 - Making changes directly on `main`
-

2. Consistency & Process (25 pts)

Goal: Work steadily, not at the last minute.

Do this

- Start working early
- Commit on multiple days using GitHub Desktop
- Open PRs before the deadline

Avoid

- First commit on deadline day
 - All work done in one session
-

3. Collaboration & Code Review (20 pts)

Goal: Collaborate using GitHub pull requests.

Open pull requests

- Open **2-3 PRs per project** via GitHub Desktop / GitHub website
- Each PR should represent a logical unit of work

Review your partner's PRs

For each PR your partner opens: 1. Open the PR on GitHub in your browser 2. Click **Files changed** 3. Click **Review changes** 4. Leave comments 5. Choose **Approve** or **Request changes** 6. Submit review

 Reviews must be submitted using **Review changes** to count.

4. Professional Git Practices (15 pts)

Goal: Use Git clearly and professionally.

Commit messages

 Good:

Add data preprocessing step
Fix visualization bug

 Bad:

```
stuff
fix
update
```

CONTRIBUTIONS.md

At the end of the project, update:

```
## Contributions
- Alice: data cleaning, modeling
- Bob: visualization, testing
```

Pair Programming (Allowed)

If you pair-program on one computer: - Use **co-authored commits** via GitHub Desktop by adding the `Co-authored-by` line in the commit message:

```
Co-authored-by: Alice <alice@school.edu>
Co-authored-by: Bob <bob@school.edu>
```

- Still open PRs and submit reviews

Automatic Flags (What to Avoid)

Flag	Meaning
NO_COMMITS	You never committed
NO_PRS	You never opened a PR
LOW_REVIEW	You never reviewed partner's code
BYPASSED_PR_WORKFLOW	Only pushed to <code>main</code>
MINIMAL_ACTIVITY	Fewer than 3 commits

If GitHub doesn't show it, **we cannot grade it**.

Full Credit Checklist

- ✓ Personal branch in GitHub Desktop
- ✓ Multiple commits
- ✓ Pull requests opened
- ✓ Partner PRs reviewed
- ✓ Work across multiple days
- ✓ Clear commit messages

Follow this workflow → **full collaboration credit**.